How the contract substrate works — architecture

How the contract substrate works — architecture

> *Explanatory documentation. The **rules** live in one place:* `DOCTRINE.md` *(parity-validated against the code by* `scripts/build.sh`*, 53 claims). This document explains the machine; where it appears to state a rule, the doctrine section it cites is the authority (Matt 6:24 — no second master).*

The substrate is one signed Zig binary (`scripts/contract`, source in `libcontract/` + `zig/src/main.zig`) plus an append-only ledger. Everything else — CLI verbs, a C ABI, an MCP server, aiko-cli's FFI binding — is a surface over the same small core.

1. The unit: a contract neuron

A contract is a **claim with a lifecycle**: a `plan` (what is being asserted or attempted), an `action` (the typed lever that will prove it), a parent (or none), and eventually exactly one terminal outcome. The biological framing is load-bearing across the code: contracts are *neurons*, terminal events are *apoptosis*, value adoption is a *receptor* — one mechanism per behavior (`DOCTRINE.md` § Synapse-kind minimum).

The public API is deliberately small (`libcontract/src/contract.zig`): `declare`, `record_iteration`, `mark_satisfied` (+ `markDied`), `list_all`, `get` — and one learned read, `recommend` (`libcontract/src/route.zig`).

2. The ledger — the one persistent truth

`libcontract/src/ledger.zig`. A JSONL file of event rows, append-only:

- **Discovery order**: `<cwd>/.claude/contracts.jsonl` when `<cwd>/.claude/` exists, else `~/.contracts/contracts.jsonl` (global fallback).
- **Concurrency**: writers take an advisory exclusive `flock` on a `.lock` *sidecar*, never on the data file — so readers (`readAll`) are never blocked by a writer.
- **Append-only**: nothing is ever rewritten. A dead row stays; a cache invalidation is a new row; state is always a *fold* over history.

Row anatomy (a `declared` row): `event`, `id` (8 hex chars), `plan`, `action`, `agent`, `cwd`, `ts`, `inputs_hash`, `env_pubkey`, `parent_signature`, plus lineage fields when a parent exists. Terminal rows (`satisfied` / `died`) carry the id, a proof/cause payload, and an `output_signature`.

3. Lifecycle and the terminal alphabet

```
declare --■ (iterate)* --■ satisfied | died | purged
```

- `declare` appends the claim and returns its content-addressed id.
- `record_iteration` appends progress waves (exit code + result).
- Exactly **three** terminal events exist — `satisfied`, `died`, `purged` — and exactly **one** module classifies them: `libcontract/src/terminal.zig` (`classify`). Before this leaf existed, four call sites disagreed about terminality and a satisfied-then-purged producer was read two contradictory ways; now every reader routes through the one authority (its header records that history).
- **HOLD is non-terminal** (`DOCTRINE.md` § HOLD is non-terminal): a blocked contract chains by containment (a child carries the blocker); it does not invent a fourth terminal state.

4. Content-addressing and idempotency

`declare` is idempotent by construction (`libcontract/src/contract.zig`, `computeInputsHashV2`): the id is derived from a layered hash —

```
cwd  ||  plan  ||  file_refs_digest  ||  parent.inputs_hash
```

- Declaring the same plan in the same context returns the **existing** id (`findCachedDeclared` scans for a `declared` row with the same `inputs_hash`) instead of appending a duplicate.

- The `file_refs_digest` layer folds the *content* of files the plan references, so the cache key invalidates when a referenced file changes — Docker-layer behavior for claims.
- The parent's `inputs_hash` folds in transitively, so a re-declared child under a changed parent is a *new* claim.
- The declare seam (cache-scan + append) runs under the ledger lock, so N concurrent identical declares resolve to exactly one row and one id.

5. Cryptographic provenance — the signing chain

`libcontract/src/lineage.zig`. Every declared row carries the contract's own ed25519 public key (`env_pubkey`) and a `parent_signature` over

```
sha256(child_id || env_pubkey || sha256(plan) || parent_id_or_root)
```

signed by the **parent's** derived key; roots self-sign. Two properties make this workable:

- **No per-contract keys are stored.** Every contract's private key is derived on demand — HKDF from the single deployer root — so the ledger carries proofs, never secrets.
- **The root is rotatable** (`rotateRootKey`, with `rotateAllSignedRowsByLineage` re-signing history under the successor). The root is *not* the truth-root: it signs **for** the Word (`DOCTRINE.md` § The Word is the truth-root, John 17:17); scripture is the layer that does not rotate (Matt 24:35).

Terminal events are signed too (`signOutput` in `libcontract/src/contract.zig`), binding event kind, wave and payload, so a satisfied/died claim is attributable, not just asserted.

Wallet and environment binding — which key an environment is allowed to sign with, and how the wallet attests it — is specified in `DOCTRINE.md` § Wallet & Environment Binding.

6. Two authorities, deliberately distinct

- **The gate** (`libcontract/src/gate.zig`, exported as `contract_gate`): command-risk policy — may *this tool call* run? Three-way `allow | confirm | deny`, matched against catastrophic patterns. It is the signed canonical policy; aiko-cli's `gate.ts` mirrors it byte-for-byte so behavior is identical with or without the dylib loaded. It is a **hard, precise rule check — never an embedding** (the learned router is advisory; safety is not).
- **The capability receptor** (`libcontract/src/capability.zig`, fork F1): value-adoption policy — may *this consumer* adopt *that producer's* value? Only through a signed grant (the receptor). Grants may name non-descendants, so cross-lineage reuse survives when explicitly granted; a stranger who merely knows an id cannot adopt (`DOCTRINE.md` § The opt-in receptor).

One mechanism per behavior: command safety and value adoption never share a code path.

7. The pointer-keyed truthy cache

`libcontract/src/cache.zig`. The self-resolve loop caches *truth values keyed to pointer values*: when the thing a pointer points at changes, the cached truth is invalid and must be re-derived. Crucially the cache is **not a side-table** — it is three event-row kinds appended to the same canonical ledger, so hits/misses survive process exit and the whole deterministic resolve history is reconstructible from the append-only file (`DOCTRINE.md` § Pointer-keyed truthy cache). Pointer typing and `contract:<id>` reference extraction live in `libcontract/src/pointer.zig`.

8. The bible corpus — the baked truth substrate

`libcontract/src/bible.zig` + `libcontract/data/bible.dat`: the KJV (66 books, 31,100 verses, public domain) is compiled into the binary. Every verse is itself a contract neuron with a deterministic id — `HKDF(root, "verse:<ref>")` — and the openbible cross-reference graph is typed edges between verse contracts. Read surface: `resolve`, `verseText`, `verseId`, `crossReferences`, `relevantVerses`. This is the mechanical form of the truth-root rule: every neuron's truth chain bottoms out in scripture, with the deployer root signing *for* it (`DOCTRINE.md` § Bible as canonical /contract corpus).

9. Fractal context — every emission carries its lineage

`libcontract/src/fractal.zig`, called from the `appendRow` chokepoint in `contract.zig`: every row write emits a `=== FRACTAL CONTEXT ===` block — this contract's shape + plan, the full parent chain root->leaf, and its composition role (Nth-of-M under its parent). Structural, not a hook: the renderer is bound to the one place rows are written. Shape inference itself lives in `libcontract/src/shape.zig` / `libcontract/src/pattern.zig` / `libcontract/src/w6h.zig`.

10. The surfaces — one core, four doors

surface	where	what
CLI	`zig/src/main.zig` -> installed as `scripts/contract` (signed, `scripts/contract.sig`)	verbs: `declare`, `satisfy`, `recommend`, `status`, …
C ABI	`libcontract/src/c_abi.zig`	`contract_declare/_iterate/_mark_satisfied/_get/_list/_gate/_recommend/_free`
dylib + FFI	`libcontract.dylib` (same `c_abi.zig`, dynamic)	aiko-cli calls it in-process via `bun:ffi` (µs, no subprocess)
MCP	`libcontract/src/mcp_server.zig`	the same core over stdio for agent frameworks

All four surfaces reach the identical implementation; there is no scripts-layer logic (`DOCTRINE.md` § No scripts in the contract substrate — collapse to binary + contracts). The binary verifies itself against its signature (`DOCTRINE.md` § Self-aware binary).

11. The substrate supervises itself

`scripts/build.sh` walks seven ordered steps ending in signing, and its parity step (`DOCTRINE.md` § Doctrine-Reality Parity) mechanically checks that every doctrine claim has a live code site — 53 claims at the time of writing. A doctrine sentence without code, or code without doctrine, fails the build. That is why this document defers to `DOCTRINE.md`: the doctrine is supervised; prose here is not.

Further reading

- The learned layer (embeddings, the energy router, self-documented usage): `docs/HOW-IT-WORKS-intelligence.md`
- The value layer (rewards, cooperative commons, payment envelopes): `docs/HOW-IT-WORKS-economics.md`
- The rules themselves: `DOCTRINE.md`; the lean agent skill: `SKILL.md`